

INTI International College Penang

School of Computing 3+0 Bachelor of Science (Hons) in Computer Science, in collaboration with Coventry University, UK 3+0 Bachelor of Science (Hons) in Software Engineering, in collaboration with Coventry University, UK

Coursework Cover Sheet

Section A - To be completed by the student.

Full Name: Chiew Kai Jun	
CU Student ID Number: P22014289	
Semester: April 2026	
Lecturer: Mr. Muhamad Nazir Bin Samsudin	
Module Code and Title: 5022CMD/5003CEM Advanced Algorithms	
Assignment No. / Title: Project Portfolio	% of Module Mark
Hand out date:	Due date: 28 June 2026
Penalties: No late work will be accepted. If you are unable to submit coursework on time due to extenuating <u>circumstances</u> you may be eligible for an extension. Please consult the lecturer.	
Declaration: I the undersigned <u>confirm</u> that I have read and agree to abide by the University regulations on plagiarism and cheating and Faculty coursework policies and procedures. I confirm that this piece of work is my own. I consent to appropriate storage of our work for plagiarism checking.	
Signature(s):  _____	

Chiew Kai Jun

P22014289

5022CMD

Portfolio

GitHub Link:

<https://github.com/p22014289-eng/CourseWork>

Turnitin Result Summary Page

Paper Title	Uploaded	Grade	Similarity
Assignment Report.docx	06/27/2026 10:46 AM +08	--	   

TABLE OF CONTENTS

Cover Page	i
Turnitin Result Summary	ii
Table of Contents	iii
Question 1: Hashing	5-9
Question 2: Divide and Conquer Algorithm	10-19
Question 3: Concurrent Process	20-25
Reflection	26
References	27

Question 1: Hashing

1) Hash Table

```
class HashTable: 1usage
    def __init__(self, size):
        self.size = size
        self.table = [None] * size
```

Screenshot 1.0: Hash Table

The hash table is a fixed size array, with each element in the array being a "bucket. The structure is built by using `self.table = [None] * size`, which sets up all buckets to be empty. An array is used because we can access it in $O(1)$ time using direct indexing. The hash function can be used to make the process of inserting and retrieving records efficient. Arrays are easier to implement than linked lists and have a smaller memory overhead. Linear Probing is used as the collision resolution technique. If a collision happens, the system looks for the next available bucket until it is found. In this way it will be possible to maintain efficient storage even if collisions occur.

2) Entity class

```
class Medicine: 11+ usages
    def __init__(self, code, name, category, price, quantity):
        self.code = code
        self.name = name
        self.category = category
        self.price = price
        self.quantity = quantity
```

Screenshot 1.1: Entity Class

System Medicine is a class to represent medicine records in the system for managing the inventory. All objects have attributes like code, name, category, price, quantity etc. The code used as identifier is the key in the hash table. The name and category indicate the type of medicine and the quantity shows the amount of medicine in stock while the price is the amount in decimal format. The features are selected to make sure that medicine data is saved in an organized and structured manner.

Attribute	Data Type	Description
code	String	Unique identifier for medicine (used as hash key)
name	String	Name of medicine
category	String	Type of medicine (e.g., tablet, syrup)
price	Float	Price of medicine in decimal form
quantity	Integer	Number of items in stock

Table 1: Entity Class Attributes

3) Command-Line System

The system has been developed with a command-line interface, which enables the user to control the records of the items in the pharmacy. It offers the capability to display all medicines, to add new medicines, search for medicines, and to compare the performance of medicine searching between a hash table and a one-dimensional array.

```

===== Pharmacy Inventory System =====
1. Display Medicines
2. Insert Medicine
3. Search Medicine
4. Edit Medicine
5. Delete Medicine
6. Compare Performance
7. Exit
Enter choice:

```

Screenshot 1.2: Main Menu

```

Enter choice: 1

--- Hash Table Structure ---
[0] M108 | FishOil | Supplement | RM35.00 | Stock:18
[1] M109 | Antacid | Tablet | RM6.50 | Stock:45
[2] M110 | HerbalTea | Supplement | RM16.00 | Stock:12
[3] Empty
[4] Empty
[5] Empty
[6] Empty
[7] Empty
[8] Empty
[9] Empty
[10] Empty
[11] Empty
[12] Empty
[13] Empty
[14] Empty
[15] Empty
[16] M101 | Panadol | Tablet | RM8.50 | Stock:50
[17] M102 | Actifed | Syrup | RM12.00 | Stock:30
[18] M103 | VitaminC | Supplement | RM18.90 | Stock:40
[19] M104 | Aspirin | Tablet | RM9.90 | Stock:20
[20] M105 | CoughMix | Syrup | RM14.50 | Stock:15
[21] M106 | ZincPlus | Supplement | RM22.50 | Stock:25
[22] M107 | Paracetamol | Tablet | RM7.90 | Stock:60

```

Screenshot 1.3: Display function

```

Enter choice: 2
Code: M111
Name: Ibuprofen
Category: Tablet
Price: 10
Quantity: 20
Added successfully. Collisions: 9

```

Screenshot 1.4: Insert function

```

Enter choice: 3
Enter Code: M105
Found: M105 | CoughMix | Syrup | RM14.50 | Stock:15

```

Screenshot 1.5: Search function

```

Enter choice: 6
Enter Code (try M101 or M999): M105

===== Performance Comparison =====
Search Key: M105
Hash Table Time : 8600 ns
Array Search Time: 1800 ns

```

Screenshot 1.6: Compare function

```
self.table = [None] * size
```

Screenshot 1.7: Hash table structure

The hash table is designed as an array of fixed size with each position being a bucket. There are buckets each containing a medicine object, or empty when no medicine is assigned. This format provides easy indexing and effective storage of data.

```
while self.table[index] is not None:
    if self.table[index][0] == key:
        return self.table[index][1]

    index = (index + 1) % self.size
```

Screenshot 1.8: Linear probing

Linear probing is used as the collision resolution technique. If there is a collision, the system looks for the next empty element in the array, and puts the element there. This will make sure that everything is put into the hash table correctly.

```
def compare_search(hash_table, medicine_list, key):
    start = time.perf_counter_ns()
    hash_table.search(key)
    hash_time = time.perf_counter_ns() - start

    start = time.perf_counter_ns()
    linear_search(medicine_list, key)
    array_time = time.perf_counter_ns() - start

    print("\n==== Performance Comparison =====")
    print("Search Key:", key)
    print("Hash Table Time :", hash_time, "ns")
    print("Array Search Time:", array_time, "ns")
```

Screenshot 1.9: compare_search function

The compare_search function is used to compare the execution time of two search operations: the hash table and linear array. It measures and compares time to demonstrate the speed of the hash table because of direct access via a hash function.

Conclusion

The one-dimensional array uses a linear search to locate records, which is the slowest search method, while the Hash Table uses a hash function to locate records directly and thus does so faster. Linear Probing technique is collision resistant and can have good search performance in case of collision. Hence, in this case, the Hash Table can be a better data structure to store and retrieve the medicine records quickly when the requirement is to fast retrieve the medicine records.

Question 2: Divide and Conquer Algorithm

1)

```
def merge_sort(arr): 3+ usages
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])    # Divide
    right = merge_sort(arr[mid:])   # Divide

    return merge(left, right)       # Conquer + Combine
```

Screenshot 2.0: Merge Sort Function

merge_sort() function is an implementation of Divide and Conquer technique. Using the transaction list, it is repeatedly split into two smaller ones, until each one is of size one. The function then sorts each sublist recursively and merges them together and returns a single sorted list.

```
def merge(left, right): 1 usage
    sorted_arr = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].transactionID < right[j].transactionID:
            sorted_arr.append(left[i])
            i += 1
        else:
            sorted_arr.append(right[j])
            j += 1

    # Add remaining items
    sorted_arr.extend(left[i:])
    sorted_arr.extend(right[j:])

    return sorted_arr
```

Screenshot 2.1: Merge Function

The merge() function is used to merge a pair of sorted sublists into one sorted list. It recognizes the transaction IDs (transaction IDs of the 1st list are considered smaller than those of the 2nd list), and adds the smaller ones to the new list, until all elements of the 2 lists have been added to the new list.

```
def binary_search(arr, target): 1 usage
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid].transactionID == target:
            return arr[mid]
        elif arr[mid].transactionID < target:
            low = mid + 1
        else:
            high = mid - 1

    return None
```

Screenshot 2.2: Binary Search Function

The `binary_search()` function looks up a transaction ID in a sorted list. It repeatedly splits the search space into two parts and only searches the appropriate part. This method is a more efficient search than the linear search method, as it requires fewer comparisons.

2)

```
class Transaction: 10+ usages
    def __init__(self, transactionID, customerName, productName, amount, transactionDate):
        self.transactionID = transactionID
        self.customerName = customerName
        self.productName = productName
        self.amount = amount
        self.transactionDate = transactionDate

    def __str__(self):
        return f"{self.transactionID} | {self.customerName} | {self.productName} | RM{self.amount} | {self.transactionDate}"
```

Screenshot 2.3: Class Transaction

The `Transaction` class is the class that holds the information for the Transactions held by the Customer. It contains attributes like `transactionID`, `customerName`, `productName`, `amount` and `transactionDate`.

The `transactionID` is used to identify the transaction and the `customerName` and `productName` are stored as strings. The `amount` is stored as a float which is used for decimal amounts, and `transactionDate` is stored as string. It makes it easy to store and control transaction information.

3)

```

transactions = [
    Transaction(transactionID: "T105", customerName: "Ali", productName: "Shoes", amount: 120.50, transactionDate: "2026-01-10"),
    Transaction(transactionID: "T101", customerName: "Sara", productName: "Bag", amount: 80.00, transactionDate: "2026-01-01"),
    Transaction(transactionID: "T110", customerName: "John", productName: "Watch", amount: 250.00, transactionDate: "2026-01-15"),
    Transaction(transactionID: "T102", customerName: "Lily", productName: "Book", amount: 40.00, transactionDate: "2026-01-02"),
    Transaction(transactionID: "T108", customerName: "Ahmad", productName: "Phone", amount: 999.00, transactionDate: "2026-01-12"),
    Transaction(transactionID: "T103", customerName: "Kumar", productName: "Laptop", amount: 3000.00, transactionDate: "2026-01-03"),
    Transaction(transactionID: "T107", customerName: "Aina", productName: "Headset", amount: 150.00, transactionDate: "2026-01-11"),
    Transaction(transactionID: "T104", customerName: "Ben", productName: "Mouse", amount: 60.00, transactionDate: "2026-01-05"),
    Transaction(transactionID: "T109", customerName: "Chen", productName: "Keyboard", amount: 120.00, transactionDate: "2026-01-13"),
    Transaction(transactionID: "T106", customerName: "David", productName: "Monitor", amount: 700.00, transactionDate: "2026-01-09")
]

```

Screenshot 2.4: Transaction Dataset

The records of transactions are stored in the system using a list (one-dimensional array). The initial data set starts with 10 pre-defined Transaction objects that are stored in an unsorted manner. This unsorted structure is important as it will be used to show sorting functionality in later steps of Merge Sort.

4a)

```

Before Sorting:
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T101 | Sara | Bag | RM80.0 | 2026-01-01
T110 | John | Watch | RM250.0 | 2026-01-15
T102 | Lily | Book | RM40.0 | 2026-01-02
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T107 | Aina | Headset | RM150.0 | 2026-01-11
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T106 | David | Monitor | RM700.0 | 2026-01-09

```

Screenshot 2.5: Transactions Before Merge Sort

```

After Sorting:
T101 | Sara | Bag | RM80.0 | 2026-01-01
T102 | Lily | Book | RM40.0 | 2026-01-02
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T106 | David | Monitor | RM700.0 | 2026-01-09
T107 | Aina | Headset | RM150.0 | 2026-01-11
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T110 | John | Watch | RM250.0 | 2026-01-15

```

Screenshot 2.6: Transactions After Merge Sort

```

Merge Sort Time: 0.000039 seconds

```

Screenshot 2.7: Merge Sort Execution Time

The transaction data is displayed initially without any order (Screenshot 2.5). Once having applied the Merge Sort algorithm, the transactions are ordered in ascending order by transactionID (Screenshot 2.6).

In Merge Sort, the list is split into smaller sublists, sorted recursively and then combined together. This will guarantee the efficient sorting performance.

The time taken to execute the algorithm as recorded in Screenshot 2.7 shows the efficiency of the algorithm; time complexity is $O(n \log n)$ which is suitable for larger datasets.

b)

```
def binary_search(arr, target): 1 usage
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid].transactionID == target:
            return arr[mid]
        elif arr[mid].transactionID < target:
            low = mid + 1
        else:
            high = mid - 1

    return None
```

Screenshot 2.8: Binary Search Function

This is a search function for a sorted list of transactionID. It begins by comparing the middle value, and then chooses to go down the left or right part of the list. This is continued until the target is located or the search space is exhausted. If transactionID is there then the system will show details of the transaction, otherwise it will show “Transaction not found”.

```
Enter choice: 3
Enter Transaction ID: T105
Found: T105 | Ali | Shoes | RM120.5 | 2026-01-10
Binary Search Time: 0.000007 seconds
```

Screenshot 2.9: Binary Search OUTPUT (EXISTING)

If a transaction exists (T105), the system is able to locate it and provide all of its information – this proves that the search function is working properly.

```
Enter choice: 3
Enter Transaction ID: T999
Transaction not found
Binary Search Time: 0.000014 seconds
```

Screenshot 2.10: Binary Search OUTPUT (NOT FOUND)

No error message is generated when looking for a non-existing transaction ID (T999), indicating proper handling of invalid input.

C)

Operation	Method	Time Complexity	Execution Time (approx)
Sorting	Merge Sort	$O(n \log n)$	0.00003 s
Searching	Binary Search	$O(\log n)$	0.00001–0.00003 s
Searching	Linear Search	$O(n)$	0.00002–0.00005 s

Table 2: Performance Comparison Table

The performance comparison illustrates that Divide and Conquer algorithms are more efficient. Merge Sort runs in $O(n \log n)$ time, as the data is repeatedly divided into smaller sublists and then is merged back again in sorted order. This helps to ensure that Merge Sort is an efficient sorting algorithm for larger data sets.

Since the search space is halved with each comparison, Binary Search has a time complexity of $O(\log n)$. Since records are added sequentially in a linear list, Linear Search has a time complexity of $O(n)$; as the size of the data set increases, Binary Search becomes a very effective method compared to Linear Search. Sorting data with Merge Sort will thus allow Binary Search to be used with a much better search performance.

5)a)

```
Enter choice: 1
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T101 | Sara | Bag | RM80.0 | 2026-01-01
T110 | John | Watch | RM250.0 | 2026-01-15
T102 | Lily | Book | RM40.0 | 2026-01-02
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T107 | Aina | Headset | RM150.0 | 2026-01-11
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T106 | David | Monitor | RM700.0 | 2026-01-09
```

Screenshot 2.11: Transactions List

This function displays all transaction records stored in the system, including transactionID, customer name, product name, amount, and transaction date. b)

```
Enter choice: 2

Before Sorting:
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T101 | Sara | Bag | RM80.0 | 2026-01-01
T110 | John | Watch | RM250.0 | 2026-01-15
T102 | Lily | Book | RM40.0 | 2026-01-02
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T107 | Aina | Headset | RM150.0 | 2026-01-11
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T106 | David | Monitor | RM700.0 | 2026-01-09

After Sorting:
T101 | Sara | Bag | RM80.0 | 2026-01-01
T102 | Lily | Book | RM40.0 | 2026-01-02
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T106 | David | Monitor | RM700.0 | 2026-01-09
T107 | Aina | Headset | RM150.0 | 2026-01-11
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T110 | John | Watch | RM250.0 | 2026-01-15

Merge Sort Time: 0.000033 seconds
```

Screenshot 2.12: Sort transactions using Merge Sort

This function sorts the transactions in ascending order based on transactionID using Merge Sort algorithm.

c)

```
Enter choice: 3
Enter Transaction ID: T105
Found: T105 | Ali | Shoes | RM120.5 | 2026-01-10
Binary Search Time: 0.000012 seconds
```

Screenshot 2.13: Search transaction using Binary Search

This is Binary Search on sorted data set to get an efficient access to a transaction.

d)

```
Enter choice: 4
Enter Transaction ID: T
Transaction not found
Linear Search Time: 0.000015 seconds
```

Screenshot 2.14: Search transaction using Linear Search

Linear Search is performed using this function which compares the records one by one. [6i\)](#)

```
def display(arr): 3+ usages
    for item in arr:
        print(item)
```

Screenshot 2.15: Display function

This function will show all the records of transactions stored in the system. It cycles through the list of transactions and prints each transaction in a user-friendly format.

```
def merge_sort(arr): 3+ usages
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])    # Divide
    right = merge_sort(arr[mid:])    # Divide

    return merge(left, right)       # Conquer + Combine
```

Screenshot 2.16: Merge Sort Function

This function is an implementation of the Merge Sort algorithm that is done in a divide and conquer fashion. It breaks the list up into smaller sublists recursively and then sorts them and combines them back together in sorted order, by transactionID.


```
def merge(left, right): 1usage
    sorted_arr = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].transactionID < right[j].transactionID:
            sorted_arr.append(left[i])
            i += 1
        else:
            sorted_arr.append(right[j])
            j += 1

    # Add remaining items
    sorted_arr.extend(left[i:])
    sorted_arr.extend(right[j:])

    return sorted_arr
```

Screenshot 2.17: Merge Function

This function combines two sorted sublists into a sorted list. Perfectly compares transactionIDs between both lists and sorts them ascending.

```
def binary_search(arr, target): 1usage
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid].transactionID == target:
            return arr[mid]
        elif arr[mid].transactionID < target:
            low = mid + 1
        else:
            high = mid - 1

    return None
```

Screenshot 2.18: Binary Search Function

This function does a Binary Search on a sorted list of transactions. Repeatedly subdivides the search space in half until it finds the target transactionID or the search is exhausted.

```
def linear_search(arr, target):
    for item in arr:
        if item.transactionID == target:
            return item
    return None
```

Screenshot 2.19: Linear Search Function

This function iterates through each transaction in turn to check whether a transaction exists. Less efficient but operates on unsorted data and is used to compare to Binary Search.

```
while True:
    print("\n==== Transaction System =====")
    print("1. Display Transactions")
    print("2. Sort (Merge Sort)")
    print("3. Binary Search")
    print("4. Linear Search")
    print("5. Exit")
```

Screenshot 2.20: Menu System

A menu-driven system that enables users to interact with the program. It can show, sort and search transaction records by various algorithms.

ii)

No additional features were implemented in this system.

iii)

```
Enter choice: 1
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T101 | Sara | Bag | RM80.0 | 2026-01-01
T110 | John | Watch | RM250.0 | 2026-01-15
T102 | Lily | Book | RM40.0 | 2026-01-02
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T107 | Aina | Headset | RM150.0 | 2026-01-11
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T106 | David | Monitor | RM700.0 | 2026-01-09
```

Screenshot 2.21: Display Function

```

Enter choice: 2

Before Sorting:
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T101 | Sara | Bag | RM80.0 | 2026-01-01
T110 | John | Watch | RM250.0 | 2026-01-15
T102 | Lily | Book | RM40.0 | 2026-01-02
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T107 | Aina | Headset | RM150.0 | 2026-01-11
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T106 | David | Monitor | RM700.0 | 2026-01-09

After Sorting:
T101 | Sara | Bag | RM80.0 | 2026-01-01
T102 | Lily | Book | RM40.0 | 2026-01-02
T103 | Kumar | Laptop | RM3000.0 | 2026-01-03
T104 | Ben | Mouse | RM60.0 | 2026-01-05
T105 | Ali | Shoes | RM120.5 | 2026-01-10
T106 | David | Monitor | RM700.0 | 2026-01-09
T107 | Aina | Headset | RM150.0 | 2026-01-11
T108 | Ahmad | Phone | RM999.0 | 2026-01-12
T109 | Chen | Keyboard | RM120.0 | 2026-01-13
T110 | John | Watch | RM250.0 | 2026-01-15

Merge Sort Time: 0.001832 seconds

```

Screenshot 2.22: Merge Sort Output

```

Enter choice: 3
Enter Transaction ID: T105
Found: T105 | Ali | Shoes | RM120.5 | 2026-01-10
Binary Search Time: 0.000007 seconds

```

Screenshot 2.23: Binary Search

```

Enter choice: 4
Enter Transaction ID: T105
Found: T105 | Ali | Shoes | RM120.5 | 2026-01-10
Linear Search Time: 0.000006 seconds

```

Screenshot 2.24: Linear Search

```

Enter choice: 5
Program ended.

Process finished with exit code 0

```

Screenshot 2.25: Exit Screen

Question 3: Concurrent process

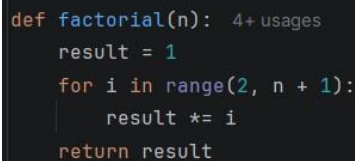
1)

Multithreading in Python is concurrent processing – not true parallel processing for CPU-bound tasks because of the Global Interpreter Lock (GIL). With multiple threads, the GIL will only let one thread run the Python bytecode.

This means that threads are not executed concurrently by multiple cores of the CPU. Instead, they alternate in performing, so the performing is concurrent and not parallel. But multithreading can also be useful for I/O intensive operations that include file operations or network requests, when the program is waiting for I/O operations, instead of doing calculations.

Thus, multithreading in Python is more concurrent python threads and not parallel python threads (For CPU bound task).

2)

A screenshot of a code editor showing a Python function definition for factorial. The code is as follows:

```
def factorial(n): 4+ usages
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Screenshot 3.0: Factorial Function

The factorial function returns the product of the numbers 1 through n (inclusive), using a loop. All of the values are multiplied to the result variable until the while loop is completed. It repeats n times, and the number of iterations is linear with the size of the input, so the time is $O(n)$.

3)

```
def run_multithreaded(): 1 usage
    t1 = time.perf_counter_ns()

    thread1 = threading.Thread(target=compute_factorial, args=("50!", 50))
    thread2 = threading.Thread(target=compute_factorial, args=("100!", 100))
    thread3 = threading.Thread(target=compute_factorial, args=("200!", 200))

    thread1.start()
    thread2.start()
    thread3.start()

    thread1.join()
    thread2.join()
    thread3.join()

    t2 = time.perf_counter_ns()

    return t2 - t1
```

Screenshot 3.1: Multithreaded Execution Function

4)

```
def run_single_thread(): 1 usage
    t1 = time.perf_counter_ns()

    factorial(50)
    factorial(100)
    factorial(200)

    t2 = time.perf_counter_ns()

    return t2 - t1
```

Screenshot 3.2: Single Thread Function

Here, the factorial of 50, 100 and 200 are computed one after the other in a single thread. All factorials are computed sequentially, each in a single thread.

To measure the execution time, the time before and after the execution run are recorded using `time.perf_counter_ns()`, with the end time being recorded after all calculations have been completed. The total time in nanoseconds is returned.

```

ROUND 1
50! completed
100! completed
200! completed
Multithreaded Time: 1879300 ns
Single Thread Time: 37900 ns
=====
ROUND 2
50! completed
100! completed
200! completed
Multithreaded Time: 793700 ns
Single Thread Time: 40100 ns
=====
ROUND 3
50! completed
100! completed
200! completed
Multithreaded Time: 675200 ns
Single Thread Time: 47700 ns
=====
ROUND 4
50! completed
100! completed
200! completed
Multithreaded Time: 714000 ns
Single Thread Time: 35800 ns
=====

```

Screenshot 3.3: Program Output (Half Output)

The program performs the factorial calculations 10 times, with a single threaded approach. The time taken to execute each round is recorded in nanoseconds.

At the end of all 10 rounds, the average time taken for the program execution is computed and displayed. This result will be compared to the result of the multithreaded version to assess performance differences.

5i)

```

def factorial(n): 4+ usages
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

```

Screenshot 3.4: Factorial Function

Factorial Function and Time Complexity

The factorial function is used to find the factorial of a number by multiplying all the integers from 1 to n using loop. The loop consists of multiplying 1 to n, from 2 to n. So the number of operations are linear in the size of the input.

Hence, the time complexity of the function is $O(n)$. The factorial function has a loop that runs n times. One multiplication and one assignment operation are done in each iteration. The growth

rate is linear as the number of operations is proportional to the size of the input. So the complexity of the factorial function is $O(n)$.

ii)

```
ROUND 1
50! completed
100! completed
200! completed
Multithreaded Time: 1879300 ns
Single Thread Time: 37900 ns
=====
ROUND 2
50! completed
100! completed
200! completed
Multithreaded Time: 793700 ns
Single Thread Time: 40100 ns
=====
ROUND 3
50! completed
100! completed
200! completed
Multithreaded Time: 675200 ns
Single Thread Time: 47700 ns
=====
ROUND 4
50! completed
100! completed
200! completed
Multithreaded Time: 714000 ns
Single Thread Time: 35800 ns
```

Screenshot 3.5: Program Output (Multithreaded vs Single Thread Performance Test) The program performs 10 rounds of multithreaded and single-threaded comparisons and saves the execution time in nanoseconds and then calculates the average.

The multithreaded version will compute the factorial of 50, 100, and 200 at the same time using three threads; the single-threaded version will compute the factorial of each one of these numbers in turn.

Based on the results, the single threaded method is more efficient than multithreading. In part due to overhead costs of creating and managing threads and in part because of Python's Global Interpreter Lock (GIL), which does not allow true parallelism for CPU-bound programs.

Conclusion

This task makes use of multithreading for I/O operations like file access and network calls, and doesn't benefit the CPU-bound task.

iii)

```
def run_multithreaded(): 1 usage
    t1 = time.perf_counter_ns()

    thread1 = threading.Thread(target=compute_factorial, args=("50!", 50))
    thread2 = threading.Thread(target=compute_factorial, args=("100!", 100))
    thread3 = threading.Thread(target=compute_factorial, args=("200!", 200))

    thread1.start()
    thread2.start()
    thread3.start()

    thread1.join()
    thread2.join()
    thread3.join()

    t2 = time.perf_counter_ns()

    return t2 - t1
```

Screenshot 3.1: Multithreaded Execution Function

In this function, three threads are created to calculate the factorials of numbers 50, 100 and 200 using `threading.Thread()`.

The time is measured as `time.perf_counter_ns()`. The start time is measured before the execution and the end time after all threads are finished.

`.start()` starts each thread and `join()` will wait for all threads to finish before calculating total time.

iv)

Discussion

This experiment shows that multithreading doesn't make much difference in the execution time. This is because the work is CPU intensive (factorial calculation), and therefore demanding constant CPU activity.

In Python, the Global Interpreter Lock (GIL) prevents multiple threads from executing Python bytecode at the same time. Because of this, although several threads are created, the threads are not truly parallel on multiple cores of CPU. Further, creating and switching threads adds additional overhead that can cause multithreading to be slower than single-threading.

Multithreading, however, will be useful when there is a wait involved, such as I/O bound operations.

Example of improvement using multithreading:

The application performance will benefit from multithreading if it has to wait for external resources, like downloading multiple files, web scraping or serving multiple client requests on a server.

Reflection

Challenges Faced and Personal Reflections

The difficulty that I had with this assignment was understanding how Hash Tables and Linear Probing function. At first, I could not figure out how collisions were dealt with. I have tested the program with various sample records and got a better understanding of the insertion and searching process in the program.

Another was the implementation of Merge Sort as it is a recursive algorithm. I took a while to get the idea of how the list is broken up into smaller sublists and then put back together in a sorted list. Following the algorithm step by step gave me a better understanding of it.

In this assignment, I got to know about Hash Tables, Merge Sort, Binary Search, Time Complexity, and Multi-threading in Python. I also acquired better programming, debugging and problem-solving ability.

In general, I was able to reinforce my knowledge of algorithms and data structures for this assignment. While there were some difficulties, the success of the tasks led me to believe that I became more confident in my Python programming skills and was also able to develop more in-depth knowledge of programming in the future.

References

1. Python Software Foundation. (2026). *threading* — *Thread-based parallelism*. Retrieved from <https://docs.python.org/3/library/threading.html>
2. Python Software Foundation. (2026). *time* — *Time access and conversions*. Retrieved from <https://docs.python.org/3/library/time.html>
3. GeeksforGeeks. (2026). *Merge Sort Algorithm*. Retrieved from <https://www.geeksforgeeks.org/merge-sort/>
4. GeeksforGeeks. (2026). *Binary Search Algorithm*. Retrieved from <https://www.geeksforgeeks.org/binary-search/>
5. GeeksforGeeks. (2026). *Linear Probing in Hashing*. Retrieved from <https://www.geeksforgeeks.org/linear-probing-in-hashing/>